

RECURSIVIDAD

¿Qué es?

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip

Caso Base:

- Es el estado simple del problema que se soluciona de forma directa sin realizar nuevas llamadas recursivas.
- Propósito: Evita el desbordamiento de memoria e iteraciones infinitas.

Caso Recursivo

- Es el bloque donde la función se llama a sí misma con parámetros reducidos.
- Propósito: Aproximar paulatinamente el problema hacia el caso base.

Funcionamiento Interno

- Pila de Llamadas (Call Stack): Estructura de datos LIFO (Last In, First Out) gestionada por el entorno de ejecución para almacenar el estado (variables locales, parámetros) de cada llamada activa.
- Fase de Crecimiento (Apilado): Cada llamada recursiva añade un marco (frame) a la pila mientras no se alcance el caso base.
- Fase de Desbobinado (Desapilado y Retorno): Al alcanzar el caso base, los resultados parciales se devuelven en orden inverso hasta completar la llamada original.

Clasificación y Características

- Según el modo de invocación:
- Directa: La función `$$` llama explícitamente a `$$`.
- Indirecta (Cruzada): La función `$$` llama a `$$`, y la función `$$` llama a `$$`.
- Según el número de llamadas:
- Simple (Lineal): Realiza una sola llamada recursiva por ejecución.
- Múltiple: Realiza dos o más llamadas recursivas en la misma ejecución (ej. Árboles).
- Según la posición de la llamada:
- En la cola (Tail Recursion): La llamada recursiva es la última instrucción ejecutada; permite optimizaciones de memoria por el compilador.
- No en la cola (Non-tail): Quedan operaciones pendientes después de que retorna la llamada recursiva.

Ventajas

- Elegancia y claridad: Código más conciso, limpio y fácil de leer para problemas intrínsecamente recursivos.
- Simplicidad en estructuras no lineales: Facilita la implementación de recorridos en árboles y grafos.
- Modelado directo: Refleja de forma natural definiciones matemáticas inducentes.

Desventajas (Palabra de enlace: presenta limitaciones como)

- Consumo de memoria: Alta sobrecarga por el uso continuo de la pila de llamadas (Stack Overflow si la profundidad es excesiva).
- Rendimiento: Generalmente más lenta que la solución iterativa equivalente debido al costo de gestión de llamadas a funciones.
- Dificultad de depuración: Rastreo complejo de errores y flujo de control en múltiples niveles.

Categoría	Casos de Uso	Algoritmos / Ejemplos Concretos
Matemáticas y Series	Cálculo de funciones definidas por inducción.	• Factorial: $n! = n \times (n-1)!$ • Serie de Fibonacci: $F(n) = F(n-1) + F(n-2)$
Estructuras de Datos	Procesamiento de estructuras jerárquicas o auto-referenciadas.	Recorrido de Árboles (Inorden, Preorden, Postorden). • Búsqueda en Grafos (DFS - Depth-First Search).
Diseño Algorítmico	Estrategia "Divide y Vencerás" (Divide and Conquer).	• Ordenamiento: Merge Sort, Quick Sort. • Búsqueda: Búsqueda Binaria.
Juegos y Optimización	Exploración del espacio de soluciones (Backtracking).	• Torre de Hanói. • Problema de las N Reinas. • Laberintos.

BIBLIOGRAFÍAS

- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). Introduction to Algorithms (4th ed.). MIT Press.
- Joyanes Aguilar, L. (2020). Fundamentos de programación: Algoritmos, estructura de datos y objetos (5.ª ed.). McGraw-Hill Education.
- Sedgewick, R., & Wayne, K. (2011). Algorithms (4th ed.). Addison-Wesley Professional.
- Weiss, M. A. (2013). Data Structures and Algorithm Analysis in C++ (4th ed.). Pearson.